

EloidaOS — Estado Atual e Caminho para Sistemas Críticos

Autor: Almir Bispo · Documento técnico · 2026

Versão: 15 tarefas + IRQs integradas + Multi-Core com tabela verdade

Sumário

- Visão Geral
- Estado Atual da Arquitetura
- IRQs Integradas ao Modelo
- Multi-Core: 1 Sistema por Núcleo
- Tabela Verdade Global (Opcional)
- O Que Já Está Pronto
- O Que Falta para Sistemas Críticos
- Roteiro de Implementação
- Considerações Finais

1. Visão Geral

O EloidaOS é um kernel cooperativo e determinístico para microcontroladores, fundamentado em uma propriedade matemática simples: **a soma de números primos distintos é única para cada combinação**. Essa propriedade permite codificar o estado completo de um sistema de tarefas interdependentes em um único número inteiro.

O documento descreve o estado atual da arquitetura — já com IRQs integradas e extensão multi-core — e o que deve ser feito para que o kernel seja utilizável em **sistemas críticos** (DO-178C, ISO 26262, IEC 62304, IEC 61508).

2. Estado Atual da Arquitetura

2.1. Princípios inegociáveis

1. **Tempo é primo** — intervalos coprimos, determinismo temporal.
2. **Task é veredito** — cada task retorna seu primo (OK) ou o (falha).
3. **MMC é correlação** — soma única identifica o estado.
4. **Desvio é política** — tabela explícita e auditável.
5. **Tudo é estático** — sem heap, sem alocação dinâmica.
6. **Tudo é verificável** — política inspecionável linha a linha.
7. **Tudo é mínimo** — kernel em centenas de bytes.

2.2. Estrutura do kernel

```
ISR → IRQ_OPS (flag)
main: IRQ() → roteia flags para tasks de IRQ → OK[tid]
main: TASKS() → tasks periódicas → OK[tid]
main: mmc() → soma todos os OK[]
main: DESVIO(soma) → escolhe task a executar
main: executa task arbitrada
```

2.3. Footprint por task

Elemento	Tamanho	Função
OK[tid]	1 byte	Veredito (primo ou o)
DONE[tid]	1 byte	Já executou ao menos 1x
COUNTS[tid]	4 bytes	Contador de ciclos
TASK_TABLE[tid]	4 bytes (flash)	Ponteiro para a task
Total	6 bytes RAM	+ 4 bytes flash

3. IRQs Integradas ao Modelo

3.1. O papel da ISR

A ISR de hardware é **mínima**: ela apenas seta uma flag em `IRQ_OPS` e limpa o flag do periférico. Se houver ação crítica (ex.: desligar PWM em sobrecorrente), ela é feita na própria ISR — mas a **validação** e o **veredito** ficam na task de IRQ.

3.2. O papel da task de IRQ

A task de IRQ é o lugar natural para:

- Ler o dado bruto do hardware;
- Validar (faixa, CRC, protocolo);
- Publicar no `SHARED` ;
- Retornar o veredito (primo ou o).

Simetria: task periódica e task de IRQ têm o mesmo contrato. Uma lê sensor por tempo; a outra lê hardware por evento. Ambas validam, publicam e retornam primo ou o.

3.3. Um único desvio de política

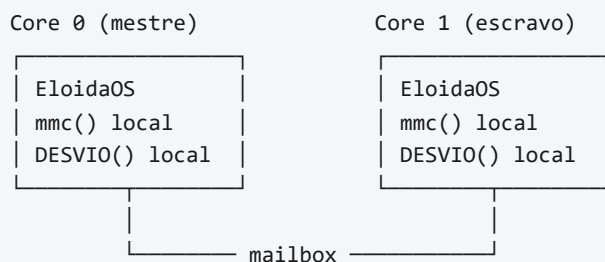
O `IRQ_TASK_MAP` **não é um desvio de política** — é um **roteamento de hardware**. Ele diz qual task trata qual IRQ, e isso é uma propriedade fixa do hardware, não da política de controle.

Há, portanto, **um único desvio de política**: o `DESVIO(soma)` , que consulta o estado completo do sistema.

4. Multi-Core: 1 Sistema por Núcleo

4.1. O modelo

Cada núcleo roda uma **instância ElويدaOS completa e independente**: seu próprio `OK[]` , `IRQ_OPS` , `mmc()` e `DESVIO()` . A comunicação entre núcleos é feita por **mailbox de hardware**, tratada como **mais um periférico**.



Vantagem: multi-core é tratado como periférico. O kernel não muda. O determinismo local é preservado.

4.2. Comunicação via mailbox

A mailbox gera uma IRQ. A IRQ seta uma flag. A flag roteia para uma task de IRQ. A task de IRQ lê a mensagem, valida, e retorna um veredito. Esse veredito entra no MMC local do núcleo receptor.

5. Tabela Verdade Global (Opcional)

5.1. Quando usar

A tabela verdade global é usada **apenas quando o usuário quer uma visão global** dos núcleos — por exemplo, quando o núcleo mestre precisa delegar tarefas ou assumir responsabilidades de núcleos falhos.

5.2. Como funciona

O mestre mantém `USER_FLAGS` com flags personalizadas que representam o estado dos núcleos remotos (OK, falha, sincronizado, etc). Uma `TABELA_GLOBAL` mapeia combinações dessas flags para ações globais.

Estado dos cores (bitmask) → ação global

CORE1_OK	CORE2_OK	CORE3_OK	MOD0_NORMAL
CORE2_OK	CORE3_OK		ASSUMIR_MOTOR
CORE1_OK	CORE3_OK		ASSUMIR_LOG
CORE3_OK			MOD0_DEGRADADO
0			FAILSAFE

Importante: a tabela verdade é **opcional**. Cada núcleo continua sendo um EloidaOS puro. As flags personalizadas e a tabela verdade são um **mecanismo de integração**, não uma camada obrigatória.

6. O Que Já Está Pronto

Item	Status	Observação
Kernel cooperativo	PRONTO	process/run/destroy
MMC por primos	PRONTO	Soma única
Desvio por tabela	PRONTO	Política auditável
IRQs integradas	PRONTO	ISR mínima + task de IRQ
Validação na task de IRQ	PRONTO	Coeso e simétrico
Multi-core como periférico	PRONTO	Mailbox tratada como IRQ
Tabela verdade global	PRONTO	Opcional, só no mestre
Footprint mínimo	PRONTO	6 bytes/task

7. O Que Falta para Sistemas Críticos

Para que o EloidaOS seja utilizável em sistemas críticos, é necessário adicionar camadas de proteção que **não quebrem os princípios** do kernel. A tabela abaixo resume o que falta.

Requisito	Status	Como implementar
Contrato de interface	FALTA	<code>TASK_WRITE_MASK[]</code> + <code>_Static_assert</code>
Snapshot transacional	FALTA	Cópia do <code>SHARED</code> antes/depois de tasks críticas
Canário de stack	FALTA	<code>CANARY_VALUE</code> no frame de cada task crítica
Watchdog por task	FALTA	<code>task_deadline[]</code> + timer de hardware
PMP / MPU	VERIFICAR	Testar se o CH32V307 implementa PMP padrão RISC-V
Log de falhas auditável	PARCIAL	<code>FAULT_LOG[]</code> já existe; adicionar timestamp e MMC
Partição em dois domínios	FALTA	Domínio rápido (ns/us) e domínio crítico (ms+)
Matriz de rastreabilidade	FALTA	Documento: requisito → mecanismo → teste

7.1. Por que PMP é a peça-chave

A proteção de memória real (partição espacial) só é possível com PMP/MPU. No CH32V307, é preciso **verificar empiricamente** se o núcleo QingKe V4F implementa o PMP padrão do RISC-V ou um mecanismo proprietário.

Se implementar, é possível proteger as tasks críticas (ms+) com regiões de memória isoladas, deixando as tasks rápidas (ns/us) sem PMP para preservar o determinismo.

7.2. Partição em dois domínios

DOMÍNIO CRÍTICO (PMP + snapshot + watchdog)

- Tasks de ms (task6+)
- Stack dedicada por task
- Comunicação via SHARED com contrato

DOMÍNIO RÁPIDO (sem PMP, sem stack dedicada)

- Tasks de ns/us (task0-task5)
- Stack compartilhada
- Comunicação via SHARED com contrato

8. Roteiro de Implementação

Fase 1 — Contrato de interface custo zero

- Definir `TASK_READ_MASK[]` e `TASK_WRITE_MASK[]`.
- Adicionar `_Static_assert` para validar em compilação.
- Documentar contrato como parte da política de desvio.

Fase 2 — Snapshot + canário ~60 bytes

- Implementar `execute_critical()` para tasks 6+.
- Implementar `execute_fast()` para tasks 0-5.
- Adicionar `FaultLog` com timestamp e MMC.

Fase 3 — Watchdog por task ~10 ciclos

- Adicionar `task_deadline[]` em tabela const.
- Implementar detecção no `SysTick_Handler`.
- Adicionar `longjmp` para deadline real.

Fase 4 — PMP ~0,17 µs/task crítica

- Testar empiricamente o PMP no CH32V307.
- Configurar regiões PMP para cada task crítica.
- Implementar `switch_to_user_mode` / `switch_to_machine_mode`.
- Capturar exception de acesso inválido.

Fase 5 — Evidência para certificação

- Gerar log de falhas em flash.
- Documentar tabela de desvio como política de controle.
- Produzir matriz de rastreabilidade (requisito → mecanismo → teste).
- Criar testes sem hardware (injeção de flags + assert).

9. Considerações Finais

O EloidaOS está, hoje, em um estado arquitetural **coeso e enxuto**:

- Um único desvio de política (`DESVIO`).
- Um roteamento de hardware (`IRQ_TASK_MAP`).
- Tasks de IRQ com o mesmo contrato das tasks periódicas.
- Multi-core como periférico, com tabela verdade opcional no mestre.
- Footprint de 6 bytes por task.

Para sistemas críticos, falta adicionar **camadas de proteção** que respeitem os princípios do kernel:

- Contrato de interface (custo zero).
- Snapshot transacional (custo ~60 bytes).
- Canário de stack (custo ~8 bytes).

- Watchdog por task (custo ~10 ciclos).
- PMP para tasks críticas (custo ~0,17 μ s/task).

A chave é **separar em dois domínios**:

- **Domínio rápido** (ns/us): sem PMP, com contrato + canário.
- **Domínio crítico** (ms+): com PMP + snapshot + watchdog.

Conclusão: o EloidaOS transforma um MCU comum em um controlador determinístico de eventos interdependentes, com custo mínimo de memória e verificabilidade total da política de controle. Com as camadas de proteção adequadas, torna-se apto a sistemas críticos — preservando o determinismo, a verificabilidade e o footprint mínimo que o definem.